



Secure Dev Labs

Software Agency | Est.2025

Student Information

Student Name	Muhammad Ahmed Raza	Batch	Batch 1
Internship Track	Ethical Hacking	Week / Task	Week 2 / Web App Mapping
Submission Date	01-05-2026	Instructor	Muhammad Saad
Tools Used	Burp Suite, Draw.io, Dvwa	File Name	Ahmed_Raza_Web_App_Mapping_Task2_Week2.pdf

1. Task Overview

Task Title: Week 2 - Web Application Mapping and Input Discovery.

Task Context: As an intern at Secure Dev Labs, my Week 2 objective is to develop a strong understanding of the “Mapping” stage in a security assessment. Before performing any vulnerability testing, I need to analyze how a web application is structured and how information flows through it. To build these skills, I will create a controlled lab environment to practice monitoring network traffic and discovering the application’s potential attack surface.

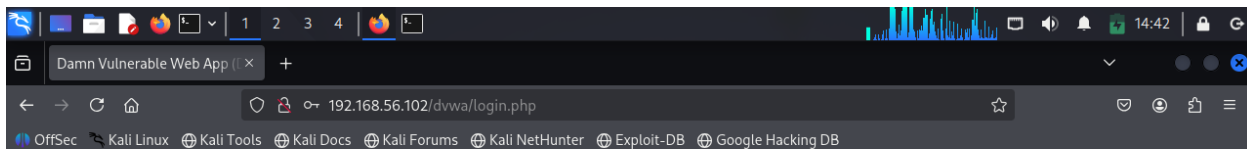
2. Objective

- **Lab Setup:** Install and configure a vulnerable web application such as DVWA or OWASP Juice Shop to create a safe and authorized practice environment.
- **Traffic Inspection:** Use Burp Suite to intercept, examine, and resend HTTP requests and responses for analysis.
- **Input Enumeration:** Identify all locations where users can provide input to the application, including forms, URL parameters, and request fields.
- **Session and Access Control:** Learn how web servers use cookies and session management to maintain user identity and enforce access permissions, including the distinction between authentication and authorization.
- **Networking Basics:** Study the DNS resolution process to understand how human-readable domain names are converted into IP addresses used by machines.

Part A:

- DVWA Installation Screenshot

```
ahemii@kali: ~  
Session Actions Edit View Help  
(ahemii@kali)-[~]  
$ git clone https://github.com/juice-shop/jiuce-shop.git  
Cloning into 'jiuce-shop' ...  
Username for 'https://github.com': ^C  
  
(ahemii@kali)-[~]  
$ git clone https://github.com/juice-shop/juice-shop.git  
Cloning into 'juice-shop' ...  
remote: Enumerating objects: 154414, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Compressing objects: 100% (2/2), done.  
Receiving objects: 100% (154414/154414), 276.54 MiB | 489.00 KiB/s, done.  
remote: Total 154414 (delta 1), reused 1 (delta 1), pack-reused 154411 (from 2)  
Resolving deltas: 100% (120346/120346), done.  
Updating files: 100% (1197/1197), done.  
  
(ahemii@kali)-[~]  
$
```

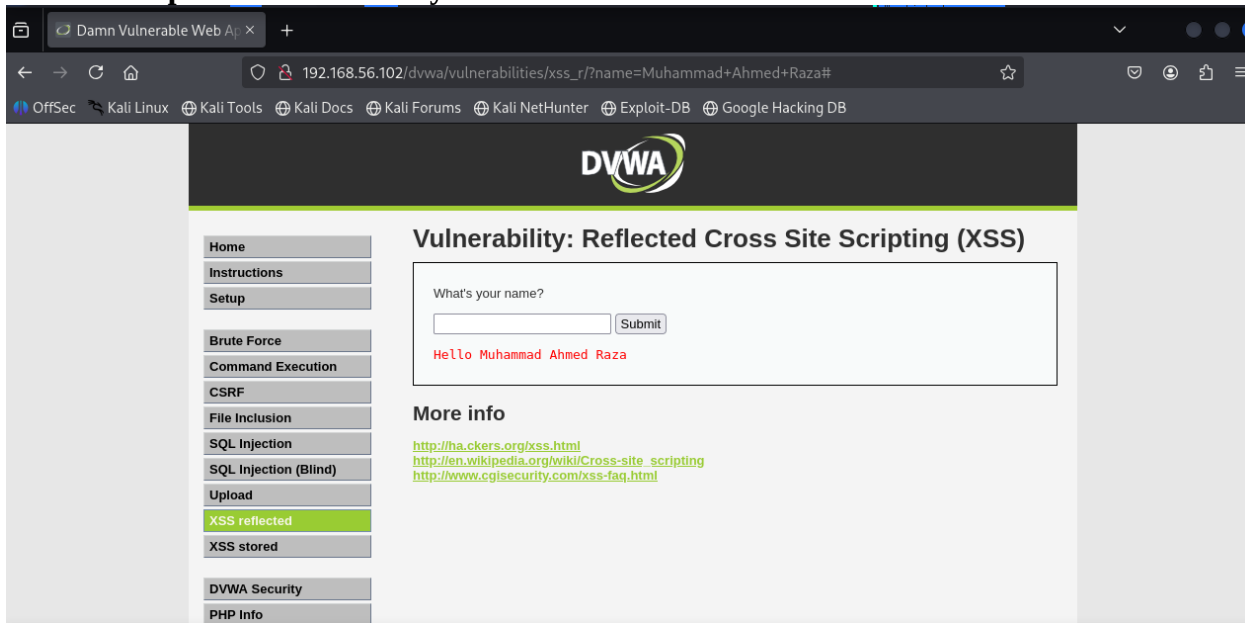


Username

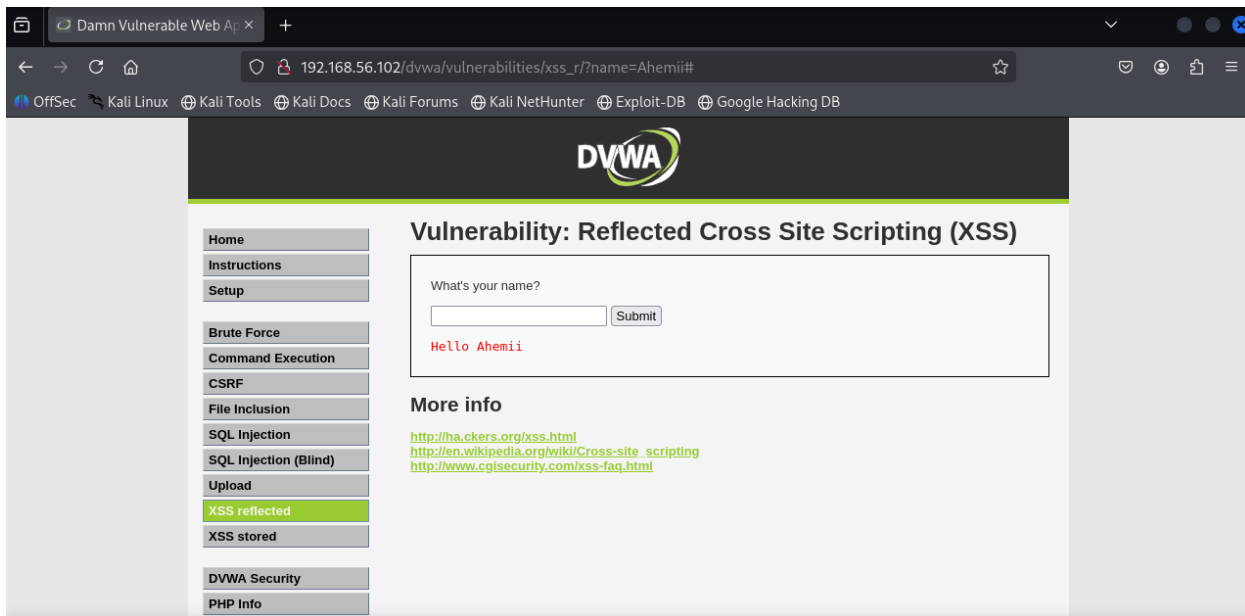
Password

Login

- **Input Fields Discovery Screenshot**



- **Map all those endpoints that are basically user controlled.**



Part B: Burp Suite

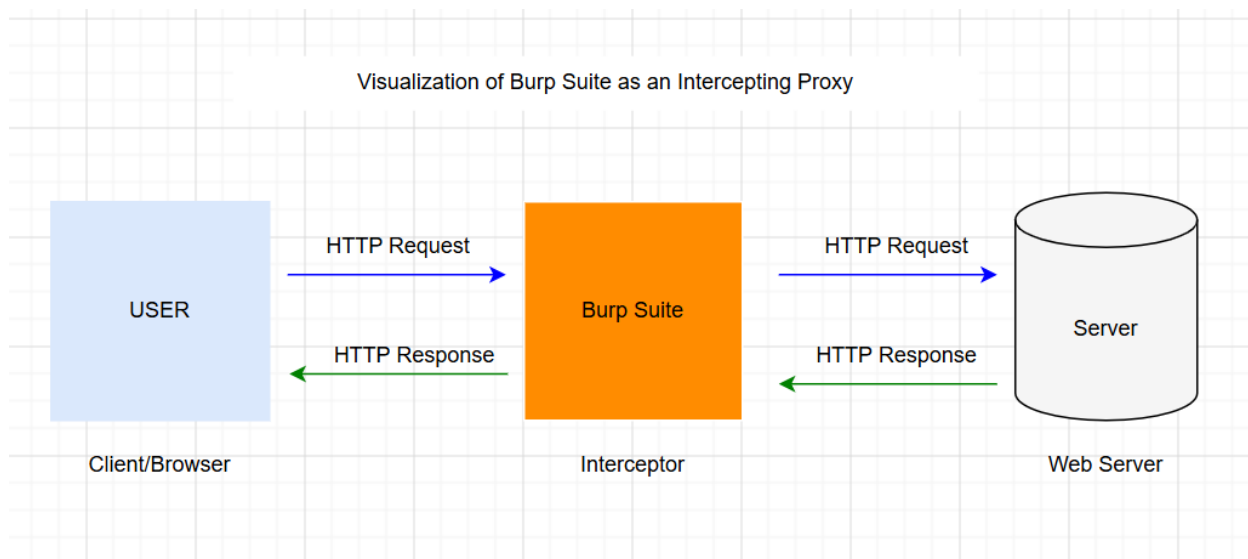
Theory:

Burp Suite is a web application security testing tool that comes pre-installed with Kali Linux. It operates as an intercepting proxy positioned between the client (the user's browser) and the server, allowing analysts to capture, inspect, and modify web traffic.

Difference B\W Burp Suite & Nmap:

Burp Suite	Nmap
It is a tool specifically used for web application security testing.	It is primarily a network discovery and security auditing tool.
It acts as an interceptor between the Client (browser) and the Server.	It scans IP addresses and ports to see what devices and services are active on a network.
It allows you to capture, inspect, and modify HTTP requests and responses	It identifies open ports, operating systems, and network services.
It focuses on finding vulnerabilities in website features like login forms, parameters, and cookies.	It focuses on the infrastructure and the server itself rather than the specific web application pages.

Visualization of Traffic Interception:



Practical

Capture your first request by using Burp Suite

The screenshot shows the Burp Suite Community Edition v2025.7.4 interface. The top menu bar includes Burp, Project, Intruder, Repeater, View, and Help. The main toolbar has buttons for Intercept on, Forward, and Drop. The status bar indicates a request to http://192.168.56.102:80. The main panel displays a table of captured requests with the following columns: Time, Type, Direction, Method, URL, Status code, and Length. The first request is a GET request to http://192.168.56.102/dvwa/index.php at 15:15:36.7 M... The right sidebar shows the Request Inspector with tabs for Request attributes, Request query parameters, Request body parameters, Request cookies, and Request headers. The Request attributes tab is selected, showing 2 attributes.

Time	Type	Direction	Method	URL	Status code	Length
15:15:36.7 M...	HTTP	→ Request	GET	http://192.168.56.102/dvwa/index.php		

Request

Pretty Raw Hex

```
1 GET /dvwa/index.php HTTP/1.1
2 Host: 192.168.56.102
3 Cache-Control: max-age=0
4 Accept-Language: en-US,en;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Referer: http://192.168.56.102/dvwa/login.php
9 Accept-Encoding: gzip, deflate, br
10 Cookie: security=high; PHPSESSID=01cd25b3128b2effa2c68d4786c11eed
11 Connection: keep-alive
12
```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 0

Request cookies 2

Request headers 10

Forward that captured request to the intercept:

The screenshot shows the Burp Suite Community Edition v2025.7.4 interface. The top menu bar includes Burp, Project, Intruder, Repeater, View, and Help. The main toolbar has buttons for Intercept on, Forward, and Drop. The status bar indicates a request to http://192.168.56.102:80. The main panel displays a table of captured requests with the following columns: Time, Type, Direction, Method, URL, Status code, and Length. The first request is a GET request to http://192.168.56.102/dvwa/vulnerabilities/xss_r/ at 15:17:46.7 M... The right sidebar shows the Request Inspector with tabs for Request attributes, Request query parameters, Request body parameters, Request cookies, and Request headers. The Request attributes tab is selected, showing 2 attributes.

Time	Type	Direction	Method	URL	Status code	Length
15:17:46.7 M...	HTTP	→ Request	GET	http://192.168.56.102/dvwa/vulnerabilities/xss_r/		

Request

Pretty Raw Hex

```
1 GET /dvwa/vulnerabilities/xss_r/ HTTP/1.1
2 Host: 192.168.56.102
3 Accept-Language: en-US,en;q=0.9
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
7 Referer: http://192.168.56.102/dvwa/index.php
8 Accept-Encoding: gzip, deflate, br
9 Cookie: security=high; security=high; PHPSESSID=01cd25b3128b2effa2c68d4786c11eed
10 Connection: keep-alive
11
12
```

Inspector

Request attributes 2

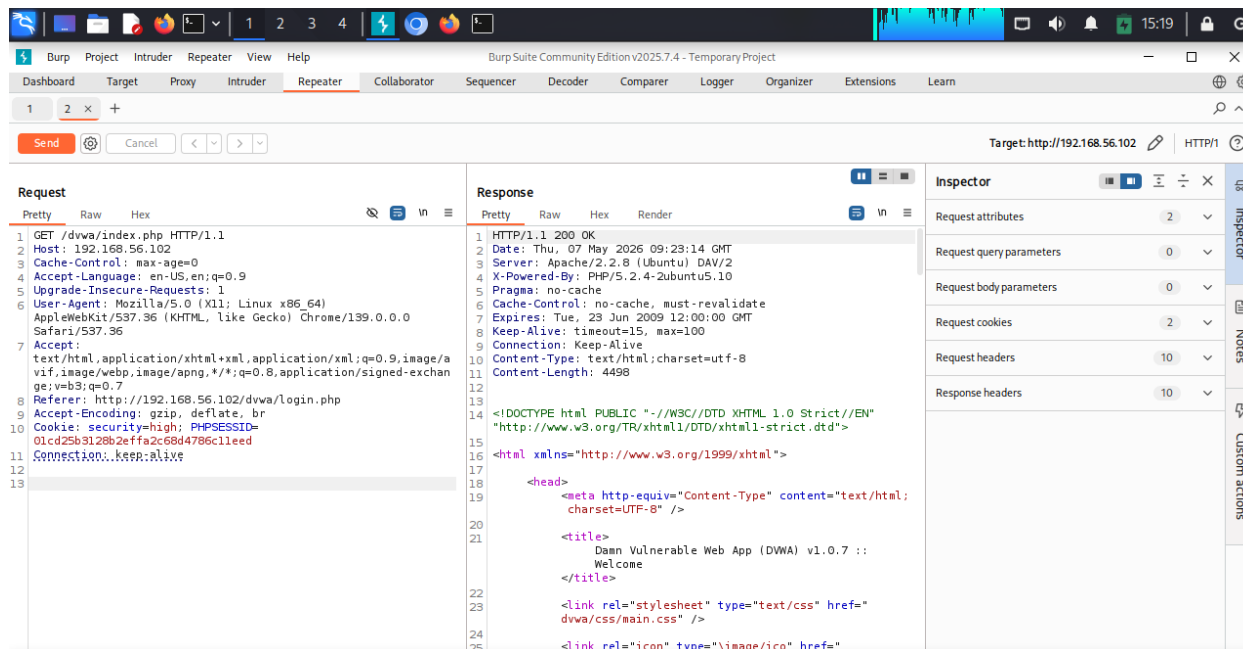
Request query parameters 0

Request body parameters 0

Request cookies 3

Request headers 9

Forward that captured request to the Repeater:



Part C: Request & Response

Request & Response in Web Applications

Request

A request is a message sent from a client, such as a web browser or mobile application, to a web server asking for a resource or service.

Example:

When a user opens a website, the browser sends a request to the server to retrieve the webpage content.

Response

A response is the message returned by the server after processing the client's request. The response usually contains the requested webpage, data, or an error message.

Example:

The webpage displayed in the browser is the server's response to the user's request.

Sender vs. Receiver

Sender (Request)

The client or browser acts as the sender by transmitting a request to the server. This can be compared to asking a question.

Receiver and Feedback (Response)

The server receives the request, processes it, and sends back a response. This is similar to providing an answer.

Example

User requests:

“Please load this webpage.” → This is a **Request**

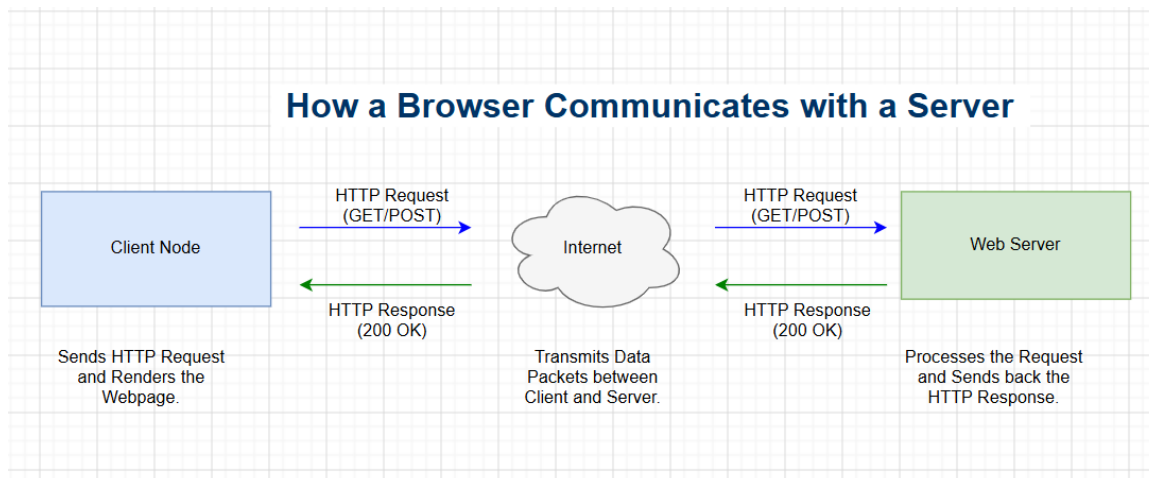
Server returns the webpage content → This is a **Response**

Common HTTP Response Codes

HTTP status codes indicate the result of a client’s request to the server.

- **200 OK** → The request was completed successfully.
- **301 Moved Permanently** → The requested page has been permanently moved to a new location.
- **400 Bad Request** → The request contains invalid syntax or incorrect data.
- **401 Unauthorized** → Authentication is required to access the resource.
- **403 Forbidden** → Access to the resource is denied.
- **404 Not Found** → The requested page or resource could not be found.
- **500 Internal Server Error** → The server encountered an unexpected problem while processing the request.

How a Browser Communicates with a Server:



Practical:

Requesting Facebook.com via Browser

The screenshot shows the Chrome DevTools Network tab. The 'Headers' sub-tab is selected. Under the 'General' section, the following information is displayed:

Property	Value
Request URL	https://www.facebook.com/index.php/
Request Method	GET
Status Code	200 OK
Remote Address	157.240.227.35:443
Referrer Policy	strict-origin-when-cross-origin

Server Response from Facebook.com

▼ Response headers	
Accept-Ch	viewport-width,dpr,Sec-CH-Prefers-Color-Scheme,Sec-CH-UA-Full-Version-List,Sec-CH-UA-Platform-Version,Sec-CH-UA-Model
Accept-Ch-Lifetime	4838400
Alt-Svc	h3=":443"; ma=86400
Cache-Control	private, no-cache, no-store, must-revalidate
Content-Encoding	zstd
Content-Security-Policy	default-src blob: 'self' https://*.fbcdn.net *.facebook.com *.fbcdn.net *.facebook.net *.whatsapp.com *.whatsapp.net;script-src *.facebook.com *.fbcdn.net *.facebook.net 127.0.0.1:* 'nonce-7GLE1XhB' blob: 'self' connect.facebook.net 'wasm-unsafe-eval' https://accounts.google.com https://*.google-analytics.com *.google.com;style-src *.fbcdn.net

Part D: Cookies and Sessions

Theory:

Cookies: The Client-Side Notebook

A cookie is a small piece of data stored directly on the **user's browser**. When a server responds to a request, it can send a Set-Cookie header telling the browser to remember certain information.

- **Persistence:** Can be set to expire when the browser closes (session cookies) or last for months (persistent cookies).
- **Storage:** Limited to about 4KB.
- **Security Risk:** Since they live on the user's machine, they can be intercepted (XSS/CSRF) unless protected by flags like Http only and secure.
-
- **Storage:** Limited to about 4KB.
- **Security Risk:** Since they live on the user's machine, they can be intercepted (XSS/CSRF) unless protected by flags like Http only and secure.

Sessions: The Server-Side Filing Cabinet

A session is stored on the **web server**. It allows the server to keep track of a specific user's state across multiple requests.

- **How it works:** The server creates a unique **Session ID**, stores it in its database (or memory), and sends that ID to the user's browser as a cookie.

- **Security:** Much safer for sensitive data (like user roles or credit card info) because the actual data never leaves the server.
- **Scaling Challenge:** If you have ten servers, they all need access to the same session data, usually requiring a centralized store like Redis

Why They Are Important in Modern Web Apps

- **Authentication & Authorization**
Without cookies/sessions, you'd have to log in on every request — which is obviously impractical.
- *User Experience (UX)*
Features like “remember me”, shopping carts, dark mode preferences all depend on them.
- *Security Layer*
Sessions help protect sensitive data (instead of exposing everything in cookies).
- *State Management*
Essential for multi-step processes (forms, checkouts, dashboards).

Define Authentication (AUTHN):

Authentication (AuthN) is the process of verifying **who** a user is. It is the digital equivalent of checking someone's ID card or passport to ensure they are exactly who they claim to be. It is often confused with *Authorization (AuthZ)*, which determines what a user is allowed to do. To keep them straight: AuthN is about identity; AuthZ is about permissions.

Define Authorization (AUTHZ): Authorization (AuthZ) is the set of permissions that determines which rooms you are allowed to enter once you're inside.

In technical terms, Authorization is the process of verifying what a specific user—already identified—can access or modify.

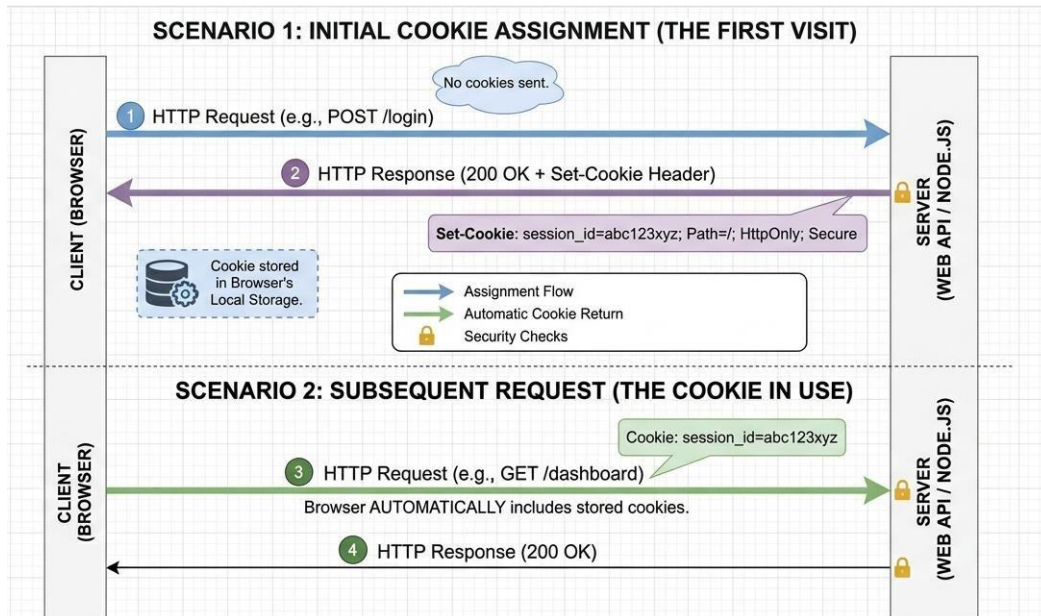
Write one difference between AUTHN and AUTHZ:

The major difference is that Authentication always happens first to verify identity, while Authorization happens second to verify permissions.

Explain why cookies/sessions are used: HTTP is stateless. Cookies and sessions provide the "memory" needed for the following:

- **Persistence:** Keeping you logged in so you don't provide credentials on every click.
- **State Management:** Tracking temporary data, like items in a shopping cart.
- **Personalization:** Remembering your settings, such as language or "Dark Mode."
- **Security:** Verifying that requests are coming from a legitimate, recognized user.

Cookie-Based Session Management in Modern Web Architecture



Practical:

Capture the HTTP request during login

Request

```
1 POST /dvwa/login.php HTTP/1.1
2 Host: 192.168.56.102
3 Content-Length: 44
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.56.102
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.56.102/dvwa/login.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=01cd25b3128b2effa2c68d4786c11eed
14 Connection: keep-alive
```

Inspector

Request attributes	2
Request query parameters	0
Request body parameters	3
Request cookies	2
Request headers	13

Identify where the credentials appear in the server response

The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. The 'Intercept' tab is active, and a request to `http://192.168.56.102:80` is intercepted. The request is a POST to `http://192.168.56.102/dwa/login.php`. The 'Request' panel shows the raw data of the request, and the 'Inspector' panel shows the decoded text of the request body, which is `username=admin&password=password&Login=Login`.

Time	Type	Direction	Method	URL	Status code	Length
15:37:467 ...	HTTP	→ Request	POST	http://192.168.56.102/dwa/login.php		

Request

```
1 POST /dwa/login.php HTTP/1.1
2 Host: 192.168.56.102
3 Content-Length: 44
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://192.168.56.102
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.56.102/dwa/login.php
12 Accept-Encoding: gzip, deflate, br
13 Cookie: security=high; PHPSESSID=01cd25b3128b2effa2c68d4786c11eed
14 Connection: keep-alive
15
16 username=admin&password=password&Login=Login
```

Inspector

Selection: 44 (0x2c)

Selected text: `username=admin&password=password&Login=Login`

Decoded from: URL encoding

Decoded text: `username=admin&password=password&Login=Login`

Request attributes: 2

Capture the HTTP request for accessing that directory

The screenshot shows the Burp Suite interface with the 'Proxy' tab selected. The 'Intercept' tab is active, and a request to `http://192.168.56.102:80` is intercepted. The request is a GET to `http://192.168.56.102/dwa/index.php`. The 'Request' panel shows the raw data of the request, and the 'Inspector' panel shows the decoded text of the request body, which is `username=admin&password=password&Login=Login`.

Time	Type	Direction	Method	URL	Status code	Length
15:45:407 ...	HTTP	→ Request	GET	http://192.168.56.102/dwa/index.php		
15:45:427 ...	HTTP	→ Request	POST	http://192.168.56.102/dwa/login.php		

Request

```
1 GET /dwa/index.php HTTP/1.1
2 Host: 192.168.56.102
3 Cache-Control: max-age=0
4 Accept-Language: en-US,en;q=0.9
5 Upgrade-Insecure-Requests: 1
6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/139.0.0.0 Safari/537.36
7 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
8 Referer: http://192.168.56.102/dwa/login.php
9 Accept-Encoding: gzip, deflate, br
10 Cookie: security=high; PHPSESSID=01cd25b3128b2effa2c68d4786c11eed
11 Connection: keep-alive
```

Inspector

Request attributes: 2

Request query parameters: 0

Request body parameters: 0

Request cookies: 2

Request headers: 10

Part E:

Theory:

Domain Name System (DNS)

The **Domain Name System (DNS)** is an internet service that converts human-readable domain names into machine-readable IP addresses. Since computers communicate using numerical IP addresses, DNS helps users access websites using easy-to-remember names instead of numbers.

Example:

Instead of remembering an IP address like `142.250.190.78`, users can simply type a domain name such as `google.com`.

DNS Request

A **DNS request**, also known as a **DNS query**, is the process where a user's device asks a DNS server to find the IP address associated with a specific domain name.

In simple terms, the device asks:

“What is the IP address for this website name?”

Example:

When a user enters a website URL into a browser, the computer sends a DNS request to locate the correct server for that website.

DNS Response

A **DNS response** is the reply sent back from the DNS server after receiving the DNS request. It contains the information required for the device to connect to the target web server.

The response typically includes the website's corresponding IP address.

Example:

If a user searches for `example.com`, the DNS server may respond with an IP address such as `93.184.216.34`, allowing the browser to establish a connection with the website server.

Compare: Domain name (human-readable) and IP address (machine-readable)

The Comparison Table

Feature	Domain Name	IP Address
Readability	Human-readable (Words and letters).	Machine-readable (Numbers and dots/colons).
Example	[www.google.com] (https://www.google.com)	142.250.190.46
Purpose	To be easily remembered and branded.	To identify a specific location on the network.
Memorability	High (Easy for humans).	Low (Hard for humans to memorize).
Stability	Static: You can move your website to a new server but keep the name.	Dynamic: Can change often if the server or host changes.

Why is DNS Important for the Internet?

The **Domain Name System (DNS)** plays a critical role in the functioning of the internet. It acts as the connection between human-readable website names and the numerical IP addresses that computers use to communicate. Without DNS, accessing websites would be far more difficult for everyday users.

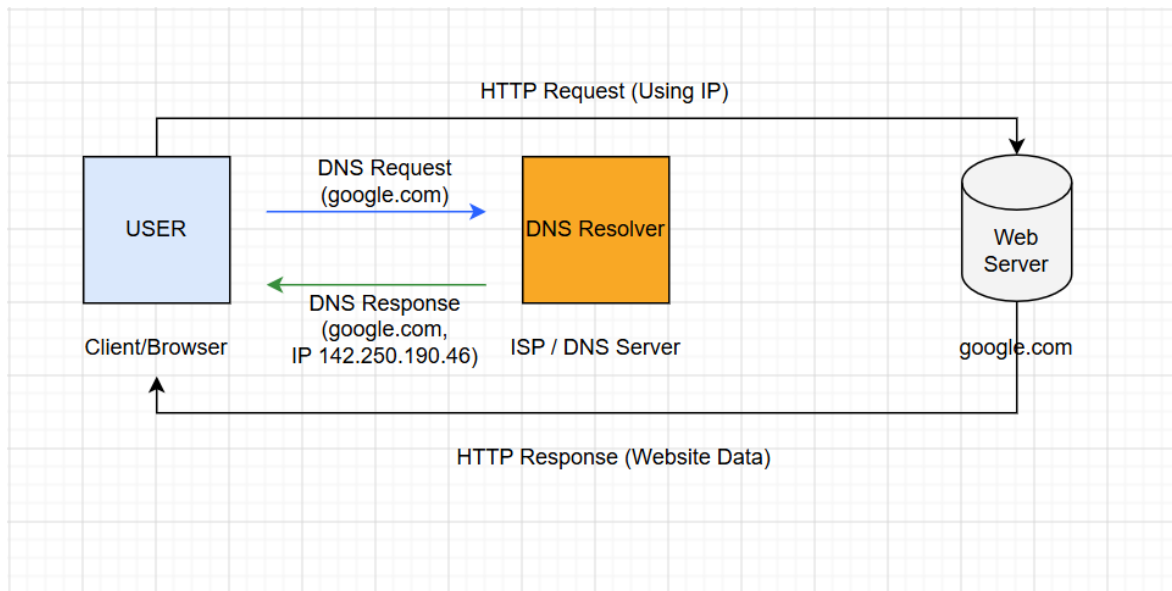
Human-Friendly Access

One of the main reasons DNS is important is that it makes the internet easy to use. Humans can easily remember words and brand names, while computers rely on numerical IP addresses.

- **Without DNS:** Users would need to memorize complex IP addresses such as 157.240.22.35 or 142.250.190.46 to access websites.
- **With DNS:** Users can simply type familiar domain names like facebook.com or google.com.

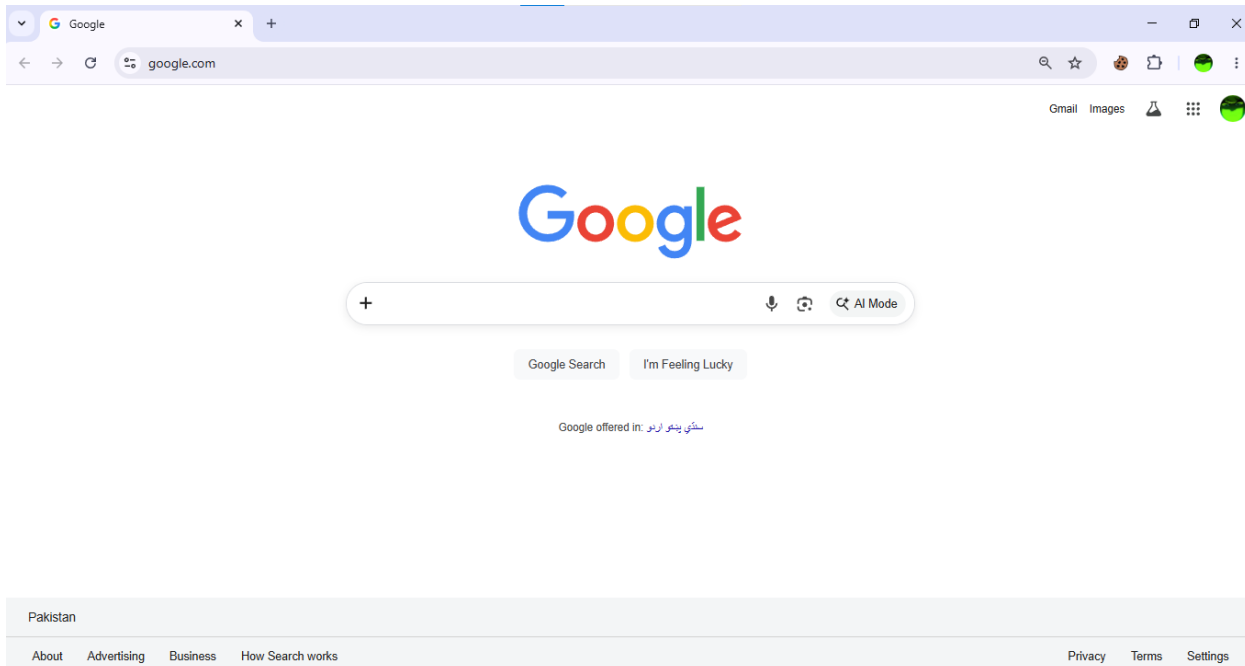
DNS improves accessibility and usability, allowing people to browse the internet conveniently without needing technical knowledge of network addresses.

DNS Lookup Process



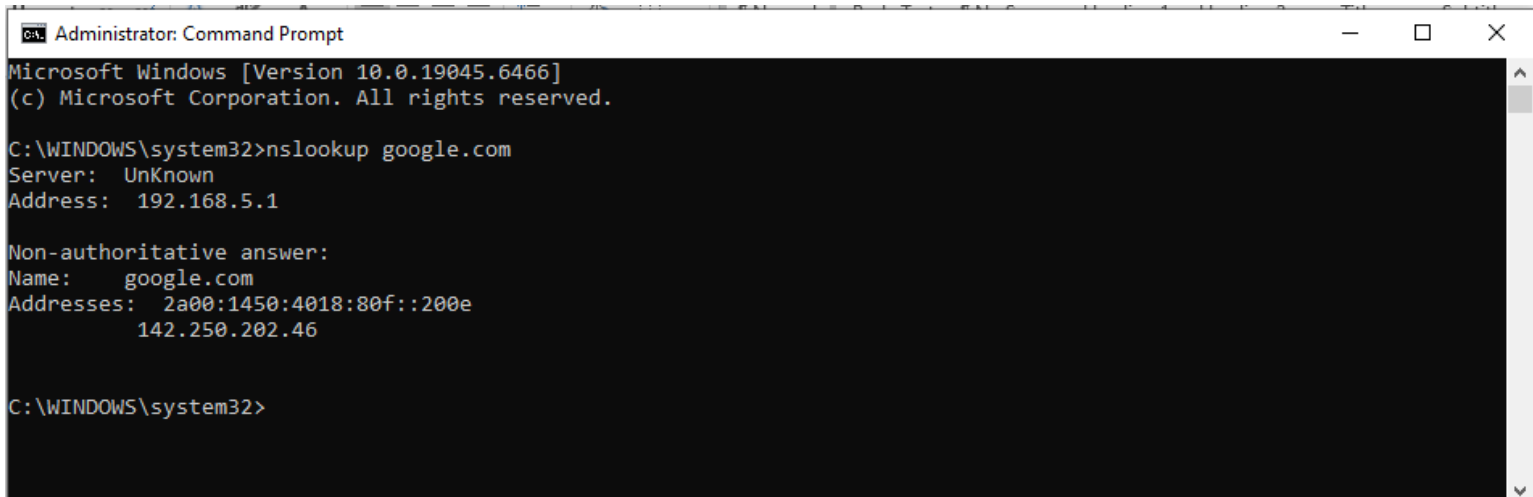
Practical:

Domain Selection: I selected google.com for this practical



DNS Request/Result:

I used the nslookup command in command prompt to observe the DNS lookup.

A screenshot of a Windows Command Prompt window titled "Administrator: Command Prompt". The window has a black background with white text. The text shows the command prompt path "C:\WINDOWS\system32>", the command "nslookup google.com", and the output of the command. The output includes the server "UnKnown" at address "192.168.5.1", a "Non-authoritative answer:", the name "google.com", and two addresses: "2a00:1450:4018:80f::200e" and "142.250.202.46". The prompt returns to "C:\WINDOWS\system32>".

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>nslookup google.com
Server:      UnKnown
Address:     192.168.5.1

Non-authoritative answer:
Name:       google.com
Addresses:  2a00:1450:4018:80f::200e
            142.250.202.46

C:\WINDOWS\system32>
```

Captured Information

Domain Name: google.com

IPv4 Returned: 142.250.202.46

IPv6 Returned: 2a00:1450:4018:80f::200e

Identification: IP Type: Both ipv4 and ipv6 addresses were identified.

Multiple IPS: Yes, the domain resolves to multiple IP addresses.